CPS 2231 Computer Programming

Chapter 4: Inheritance in Java

Dr. Hamza Djigal

Assistant Professor,

Department of Computer Science,
Wenzhou-Kean University

dhamza@kean.edu

Semester: Fall 2025

October 5, 2025

温州肯恩大学
WENZHOU-KEAN UNIVERSITY

Objectives

- The need of Inheritance in **Java**
- Relationship between **Classes**
- Class members that are inherited by a **derived class**

Introduction to Inheritance

Definitions

- **Inheritance** in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object.
- The idea **behind inheritance** in Java is that you can create new classes that are built upon existing classes.
- **Inheritance** represents the **IS-A relationship** which is also known as a parent-child relationship.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

When you inherit from an existing class, you can reuse methods and fields of the parent class.

Introduction to Inheritance

Syntax of Inheritance in Java

```
1 class SubClass-name extends SuperClass-name{
2
3 //methods and fields
4
5 }
```

- The **extends** keyword indicates that you are making a new class that derives from an existing class.
- A class which is inherited is called a parent or superclass, and the new class is called child or subclass.

Need to inherit classes in Java

Classes **Programmer** and **Manager** have common properties: **name, address, phoneNumber, and experience**

Programmer

name
address
phoneNumber
experience
programmingLanguages
writeCode()

```
class Programmer {
    String name;
    String address;
    String phoneNumber;
    float experience;
    String[] programmingLanguages;
    void writeCode() {}
}
```

Manager

name
address
phoneNumber
experience
teamSize
reportProjectStatus()

```
class Manager {
    String name;
    String address;
    String phoneNumber;
    float experience;
    int teamSize;
    void reportProjectStatus() {}
}
```

Figure 1: Two classes with common properties

Need to inherit classes in Java

Identify the common properties and put them in another class named Employee.

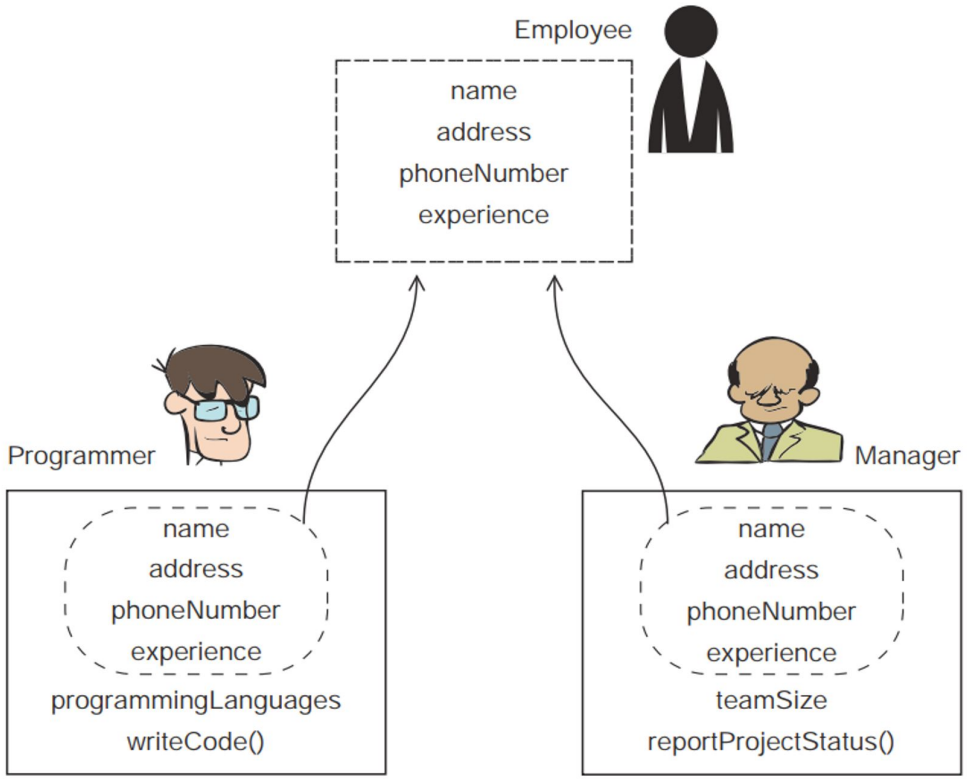
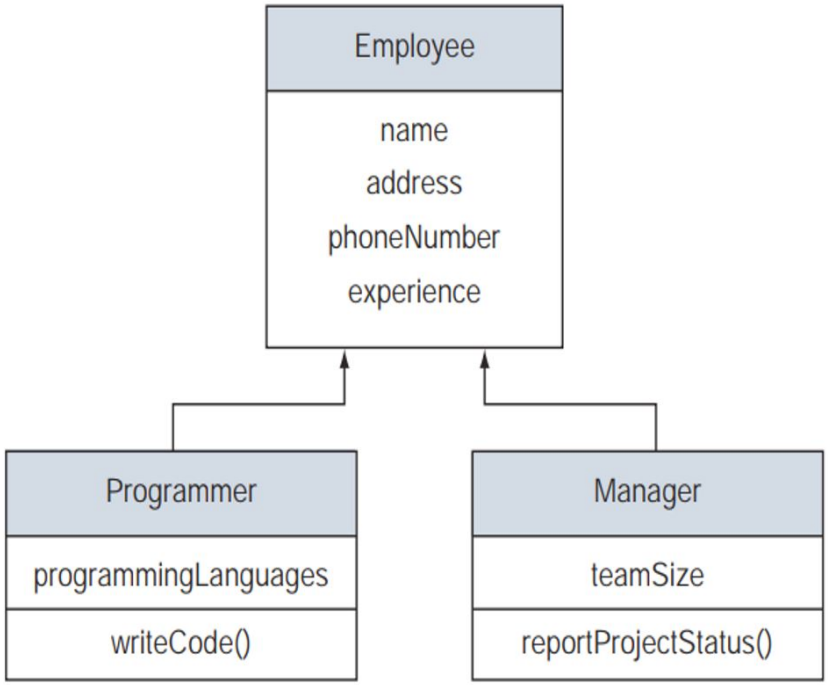


Figure 2: Identify common properties

Need to inherit classes in Java




```
class Employee {
    String name;
    String address;
    String phoneNumber;
    float experience;
}
class Programmer extends Employee {
    String[] programmingLanguages;
    void writeCode() {}
}
class Manager extends Employee {
    int teamSize;
    void reportProjectStatus() {}
}
```

Figure 3: Class **Programmer** and **Manager** extends Employee

Need to inherit classes in Java

Differences in the size of the classes A**stronaut, Doctor, Programmer, and Manager**, both with and without inheriting from the class Employee

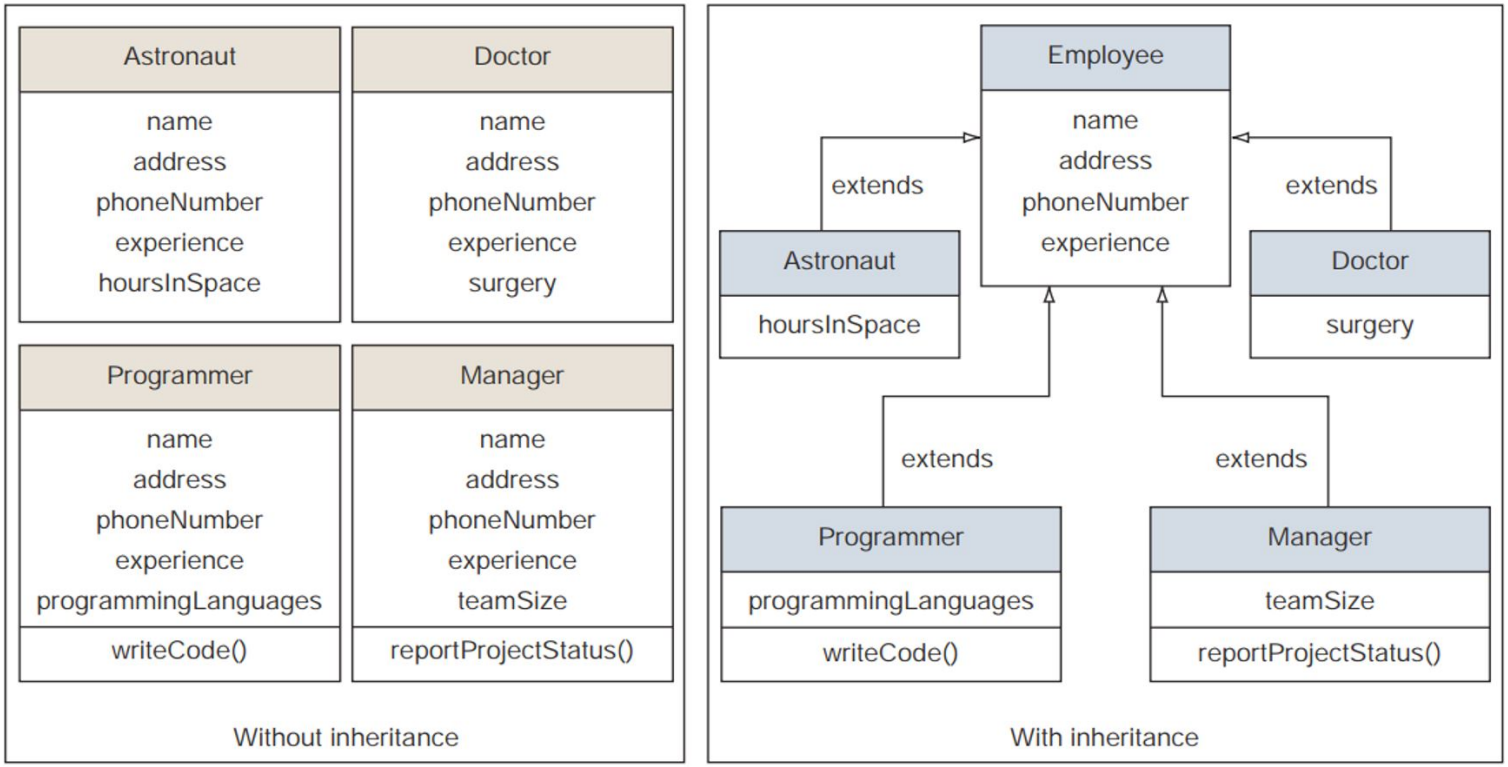


Figure 4: With and Without Inheritance

Need to inherit classes in Java

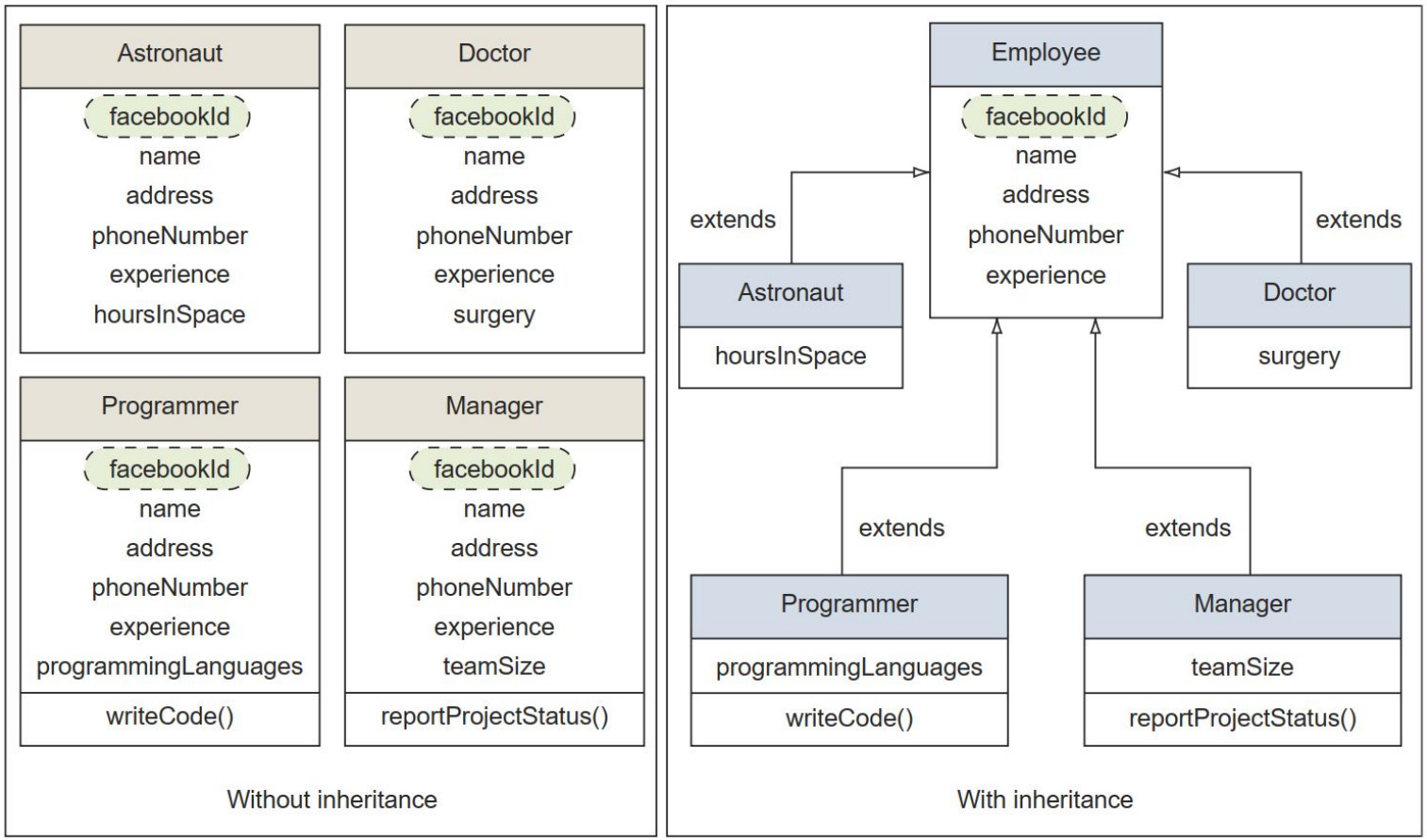


Figure 5: Extending a class offers multiple benefits.

Need to inherit classes in Java

Extensibility

Code written for a base class can also work with its subclasses.
Example: a method that accepts Employee can also accept Manager, Doctor, or any future subclass.
This makes the system easier to extend without rewriting existing code.

Reuse of Tried-and-Tested Code

Subclasses can reuse methods and fields from the base class.
No need to "reinvent the wheel", reduces duplication and increases reliability.

Focus on Specialized Behavior

Developers can concentrate on the unique features of a subclass (e.g., Doctor's surgery, Programmer's writeCode) while reusing general code from the parent class (Employee).

Need to inherit classes in Java

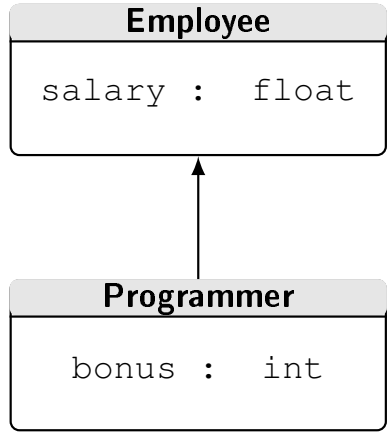
Logical Structure and Grouping

Inheritance naturally organizes classes into a hierarchy.
Example: Astronaut, Doctor, Programmer, and Manager all belong to the group of Employee.
This improves code readability and maintainability.

```
1  class Employee {
2      String name;
3      String address;
4      String phoneNumber;
5      float  experience;
6  }
7  class Programmer extends Employee {
8      String[] programLang;
9      void writeCode() {}
10 }
11 class Manager extends Employee {
12     int  teamSize;
13     void reportProjectStatus() {}
14 }
```

Inheritance Example

- **Programmer** is the **subclass** and **Employee** is the **superclass**.
- The relationship between the two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.




```
1  class Employee {
2    float salary = 40000;
3  }
4  class Programmer extends Employee {
5    int bonus = 10000;
6    public static void main(String[] args)
         {
7      Programmer p = new Programmer();
8      System.out.println("Programmer
           salary is: " + p.salary);
9      System.out.println("Bonus of
           Programmer is: " + p.bonus);
10   }
11 }
```

Types of Inheritance in Java

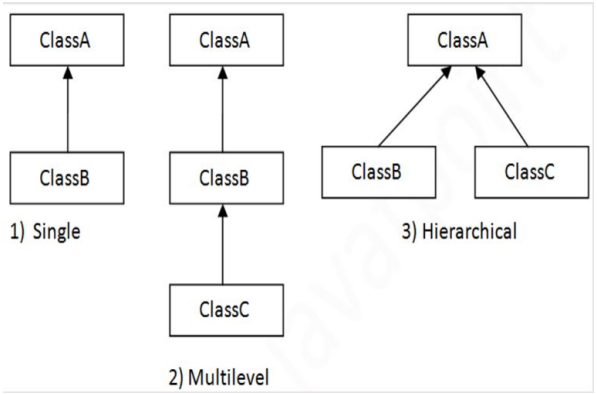


Figure 6: Type of Inheritance in Java

1. Single Inheritance

```
1  class Parent {
2      void display() { System.out.println("Parent");
           }
3  }
4  class Child extends Parent {
5      void show() { System.out.println("Child"); }
6  }
```

Types of Inheritance in Java

2. Multilevel Inheritance

```
1 class Grandparent {
2     void greet() { System.out.println("Hello"); }
3 }
4 class Parent extends Grandparent { }
5 class Child extends Parent { }
```

3. Hierarchical Inheritance

```
1 class Parent {
2     void display() { System.out.println("Parent");
          }
3 }
4 class ChildA extends Parent { }
5 class ChildB extends Parent { }
```

Why Multiple Inheritance is not Supported in Java (with classes)

Reason: To avoid ambiguity and complexity (**Diamond Problem**). If two parent classes define the same method, the compiler cannot decide which one to inherit.

Example: Diamond Problem (Not Allowed in Java)

```
1  class A {
2      void show() { System.out.println("From A"); }
3  }
4
5  class B {
6      void show() { System.out.println("From B"); }
7  }
8
9  // ERROR: Java does not allow this
10 class C extends A, B { }  // Ambiguity: which show
       () method to call?
```

Why Multiple Inheritance is not Supported in Java (with classes)

Solution: Java supports multiple inheritance through **interfaces** (We will learn it later).

Correct Example with Interfaces

```
1  interface A { void show(); }
2  interface B { void show(); }
3
4  class C implements A, B {
5      public void show() {
6          System.out.println("Resolved in C");
7      }
8  }
```

Association in Java

Definition:

- **Association** defines the relationship between two classes.
- It can be **one-to-one, one-to-many, many-to-one** or many**-to-many**.
- It shows how objects communicate and use each other's functionality.

Examples

One-to-One: A Person has only one Passport

```
1 class Person { Passport passport; }
2 class Passport { }
```

One-to-Many: A College has many Students

```
1 class College { List<Student> students; }
2 class Student { }
```

Association in Java

Examples

Many-to-One: Many Cities belong to one State

```
1 class City { State state; }
2 class State { }
```

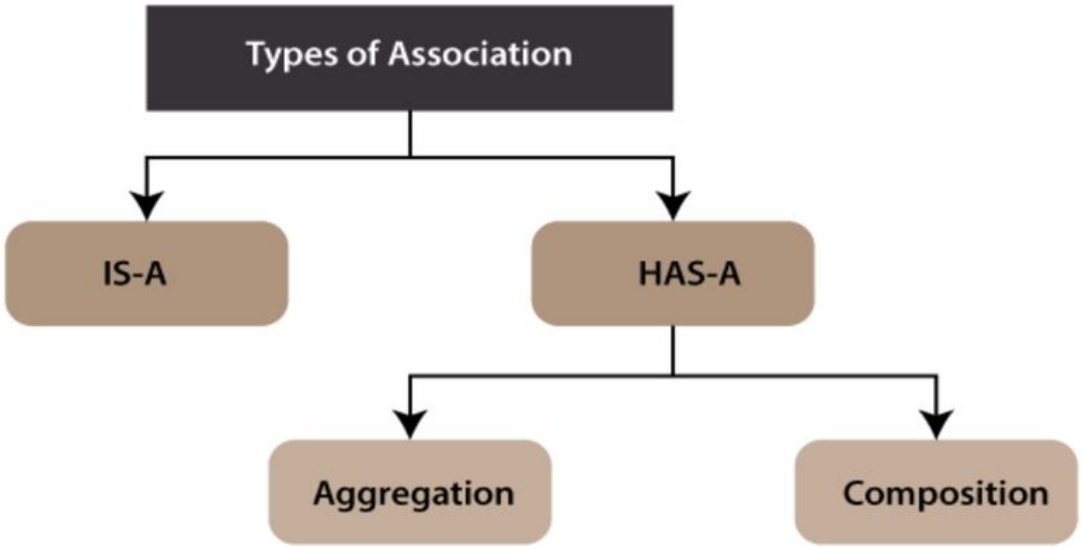
Many-to-Many: Students and Teachers

```
1 class Student { List<Teacher> teachers; }
2 class Teacher { List<Student> students; }
```

Type of Association in Java

In Java, two types of Association are possible:

- **IS-A Association:** referred to as Inheritance.
- **HAS-A Association**
  - Aggregation
  - Composition



Type of Association in Java

Aggregation in Java

- **Aggregation:** means putting objects together to make a bigger object.
- In Java, the **Aggregation** association defines the HAS-A relationship.
- Aggregation is close dependency.

Example: A car cannot move without a wheel. The **wheel** object can exist without the car object, which proves to be an aggregation relationship. **A car HAS-A wheel.**

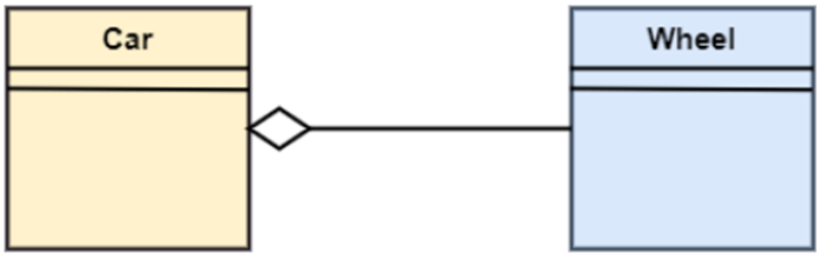


Figure 7: Aggregation example in java

Type of Association in Java

Aggregation in Java, Example

```
1  // Wheel class can exist independently
2  class Wheel {
3      void rotate() {
4          System.out.println("Wheel is rotating");
5      }
6  }
7  // Car class HAS-A Wheel (aggregation)
8  class Car {
9      Wheel wheel;  // Aggregation: Car "has a" Wheel
10     Car(Wheel wheel) {//Constructor accepts a Wheel
11         this.wheel = wheel;
12     }
13     void drive() {
14         System.out.println("Car is driving");
15         wheel.rotate();  // Using the wheel object
16     }
17 }
```

Type of Association in Java

Aggregation in Java, Example

```
1  // Wheel class can exist independently
2  class Wheel {
3      ........
4  }
5  class Car {
6      ........
7  }
8  public class TestAggregation {
9      public static void main(String[] args) {
10         Wheel w = new Wheel();//Wheel exists
               independently
11         Car c = new Car(w);//Car aggregates the
               wheel
12         c.drive();
13     }
14 }
```

Type of Association in Java

Composition in Java

Composition: Is a restricted form of the Aggregation where the entities are **strongly dependent** on each other. Unlike Aggregation, Composition represents the **part-of relationship**.

- It depicts dependency between a **composite (parent)** and its **parts (children)**, which means that if the composite is discarded, so will its parts get deleted.
- It exists between similar objects.

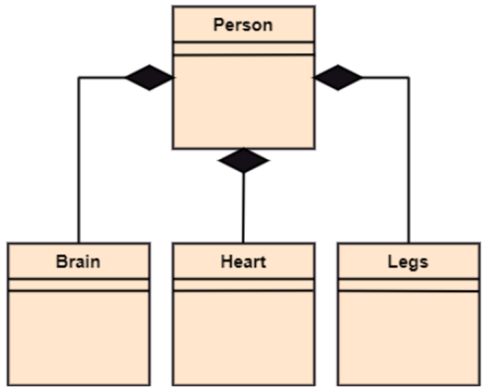


Figure 8: Composition example in java

Type of Association in Java

Composition in Java, Example

```
1  // Part classes (cannot exist independently in this
       context)
2  class Brain {
3      void think() {
4          System.out.println("Brain is thinking");
5      }
6  }
7  class Heart {
8      void beat() {
9          System.out.println("Heart is beating");
10     }
11 }
12 class Legs {
13     void walk() {
14         System.out.println("Legs are walking");
15     }
16 }
```

Type of Association in Java

Composition in Java, Example

```
1  //Person class COMPOSES Brain, Heart, and Legs
2  class Person {
3      private Brain brain;
4      private Heart heart;
5      private Legs legs;
6  //Person creates its parts internally (composition)
7      Person() {
8          this.brain = new Brain();
9          this.heart = new Heart();
10         this.legs = new Legs();
11     }
12     void live() {
13         brain.think();
14         heart.beat();
15         legs.walk();
16         System.out.println("Person is alive");
17     } }
```

Type of Association in Java

Composition in Java, Example

```
1  // Wheel class can exist independently
2  class Brain {........}
3  class Heart {........}
4  class legs{........}
5  class Person {........}
6  public class TestComposition {
7      public static void main(String[] args) {
8          Person p = new Person();
9          p.live();
10     }
11 }
```

Note: Brain, Heart, and Legs exist only inside a Person object. When the Person object is destroyed, its parts are destroyed too.

Access Modifiers and Inheritance in Java

- **Access modifiers** determine which base class members a derived class can inherit.
- A derived class can inherit only what it can **see**.
- **All non-private members** of the base class are inherited.

Accessibility Levels Inherited by Derived Classes

1. Default (no modifier): Accessible in derived class only if base and derived classes are in the **same package**.

```
1 package package1;
2 class Base {
3   int defaultVar = 10;// default access
4 }
5 class Derived extends Base {
6   void show() {System.out.println(defaultVar);//OK
        if same package
7                 }
8 }
```

Access Modifiers and Inheritance in Java

Accessibility Levels Inherited by Derived Classes

2. Protected: Accessible to all derived classes, even if they are in **different packages**.

```
1  package package1;
2  class Base {
3      protected int protectedVar = 20;
4  }
5  package package2;
6  import package1.Base;
7  class Derived extends Base {
8      void show() {
9          System.out.println(protectedVar); //
               Accessible even in different package
10     }
11 }
```

Access Modifiers and Inheritance in Java

Accessibility Levels Inherited by Derived Classes

3. Public: Accessible to all other classes, everywhere.

```
1 class Base {
2     public int publicVar = 30;
3 }
4 class Derived extends Base {
5     void show() {
6         System.out.println(publicVar); //
              Accessible anywhere
7     }
8 }
```

Which base class members aren't inherited by a derived class?

- **Private Members**: Derived class cannot access private fields or methods of the base class directly.

Private Members Not Inherited

```
1  class Parent {
2      private int secret = 42; //private not
           inherited
3  }
4
5  class Child extends Parent {
6      public void show() {
7          //System.out.println(secret); //ERROR:
               secret not accessible
8      }
9  }
```

Which base class members aren't inherited by a derived class?

- **Default Access Members (Package-Private)**: If the base class and derived class are in *different packages*, default members of the base class are not inherited.

```
1  // File: package1/Parent.java
2  package package1;
3  public class Parent {
4      int data = 100; //default (package-private)
5  }
6  //File: package2/Child.java
7  package package2;
8  import package1.Parent;
9  class Child extends Parent {
10     void display() {
11         //System.out.println(data); //ERROR: data
               not accessible outside package
12     }
13 }
```

Which base class members aren't inherited by a derived class?

- **Constructors of the Base Class**
  - Constructors are not inherited.
  - A derived class can call a base class constructor using the `super` keyword (explicitly or implicitly).

```
1  class Parent {
2      Parent() {
3          System.out.println("Parent constructor");
4      }
5  }
6
7  class Child extends Parent {
8      Child() {
9          super(); // must call explicitly or
               implicitly
10         System.out.println("Child constructor");
11     }
12 }
```

Question /Answer: Inheritance in Java

Which of the following statements about inheritance is true?

A. Inheritance allows objects to access commonly used attributes and methods.

B. Inheritance always leads to simpler code.

C. All primitives and objects inherit a set of methods.

D. Inheritance allows you to write methods that reference themselves.

Answer & Explanation

Correct Answer: A

- **Option A: True** — Inheritance improves code reusability by allowing subclasses to reuse commonly used attributes and methods from parent classes.
- **Option B: False** — Inheritance *may* simplify code, but poor design can also make it more complex.
- **Option C: False** — All *objects* inherit methods from `java.lang.Object`, but **primitives do not**.
- **Option D: False**: Methods referencing themselves is recursion, not inheritance.

Practice

1. Create a class **Employee** and a class **Programmer** which have the following fields:
   - The data members of the **Employee** are: `Name (string)`, `salary (double)`, and a `programmer (string)`.
   - The data member of the **Programmer** class is: `phone number`.
2. Create a public class **AssociationExample** which contains the `main` class.
3. In the `main` class, create instances of **Employee** and **Programmer** to set and print the given values.
4. Implement the example of **slide 7**.

The `super` Keyword in Java

super Keyword

`super` is a reference variable that refers to the immediate parent class object. When a subclass is instantiated, the parent class is also instantiated and can be accessed using `super`.

Advantages:

- Simplifies constructor chaining with `super()`.
- Provides access to overridden methods of the parent.
- Accesses hidden fields in the parent class.
- Promotes code reusability and reduces redundancy.
- Maintains hierarchical relationships in inheritance.

Uses:

- Refer to parent class instance variables.
- Invoke parent class methods.
- Call parent class constructor with `super()`.

super with Instance Variables

Use Case: super can be used to refer to the immediate parent class instance variable when it is hidden by a child class variable.

Example

```
1  //Parent class
2  class Animal {
3      String color = "white";
4  }
5  // Child class
6  class Dog extends Animal {
7      String color = "black";
8      void printColor() {
9        System.out.println(color); //Dog color
10       System.out.println(super.color);//AnimalColor
11     }
12 }
```

Using `super` to Call Parent Method

The `super` keyword can be used to call the parent class method when it is **overridden** in the child class.

Example

```
1  class Animal {//Parent class
2      void eat() { System.out.println("eating..."); }
3  }
4  class Dog extends Animal {//Child class
5      void eat(){System.out.println("eating bread");}
6
7      void bark(){System.out.println("barking..."); }
8
9      void work() {
10         super.eat();//calls parent class method
11         bark();
12     }
13 }
```

Using super to Call Parent Method

```
1  class Animal {// Parent class
2      ........
3  }
4  class Dog extends Animal {// Child class
5      ........
6  }
7  // Main class
8  public class Main {
9      public static void main(String args[]) {
10         Dog d = new Dog();
11         d.work();
12     }
13 }
```

Output

```
eating...
barking...
```

super() to Invoke Parent Constructor

- The `super()` keyword can be used inside a subclass constructor to explicitly **call the parent class constructor**.
- It ensures that the parent object is properly initialized before the child adds its own initialization.

```
1  class Animal {
2      Animal() {
3          System.out.println("animal is created");
4      }
5  }
6  class Dog extends Animal {
7      Dog() {
8          super();    // calls parent constructor
9          System.out.println("dog is created");
10     }
11 }
```

super() to Invoke Parent Constructor

```
1  class Animal {
2      Animal() {System.out.println("animal is created
           "); }
3  }
4  class Dog extends Animal {
5      Dog() {super();    //calls parent constructor}
6  }
7  public class Main {
8      public static void main(String args[]) {
9          Dog d = new Dog();
10     }
11 }
```

Output

animal is created
dog is created

Real-Time Use of super

```
1  class Person {
2      int id;  String name;
3      Person(int id, String name) {
4          this.id = id;
5          this.name = name;
6      }
7  }
8  class Emp extends Person {
9     float salary;
10    Emp(int id, String name, float salary) {
11        super(id, name);//reusing parent constructor
12        this.salary = salary;
13     }
14    void display() {
15       System.out.println(id+" "+name+" "+salary);
16     }
17 }
```

Real-Time Use of super

```
1  class Person {
2      .....
3  }
4  class Emp extends Person {
5     .....
6  }
7  public class Main {
8      public static void main(String[] args) {
9          Emp e1 = new Emp(1, "ankit", 45000f);
10         e1.display();
11     }
12 }
```

Output

1 ankit 45000.0

Practice

- Write a program in Java for a hierarchy **Employees** of a company as follows:
- The base class should be **Employee**, with subclasses **Manager**, **Developer**, and **Programmer**.
- Each subclass should have properties such as **name**, **address**, **salary**, and **job title**.
- Implement methods for **calculating bonuses (bonusCal)**, **generating performance reports**, and **managing projects**.

Note:

- The bonus of **Manager**, **Developer**, and **Programmer** can be 0.5%, 2%, and 1% respectively.
- The performance can be: **Bad, Good, Excellent**.
- Managing project is the project handled by the employee: e.g., developer is coding in SIS information using Java.